ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

AI Agents Development & Agentic Automation

Eng. Fatma Elraey

Lab 3 — Build an n8n Agent Workflow with Gemini and Business Tools

Field	Detail
Instructor	Fatma Elraey
Course	AI Agents Development (3-Day Course)
Session	Day 3 of 3 — Lab, 3 Hours
Focus	Hands-on visual orchestration: create a webhook-driven agent, connect Gemini, add memory, use Google Sheets/Slack/Gmail-style tools, and add a safe review path for sensitive actions.

Before You Start

You need access to an n8n workspace (cloud or local) and a Gemini credential. Slack, Gmail, and Google Sheets are optional but recommended for the full lab; if you do not have them, use mock data or an HTTP Request node to preserve the same workflow design.

Today's Lab Plan

1. Create the trigger and AI Agent
2. Connect Gemini and system instructions
3. Add short-term memory
4. Add Google Sheets lookup as a tool
5. Connect a notification/delivery tool
6. Test webhook, error, and approval behavior

✓ QUICK CHECK — Before You Start

Q1. What is the main benefit of n8n for this day?

Answer: *Visual event-driven orchestration across AI and business integrations.*

Q2. Should an AI Agent node receive unrestricted tools?

Answer: *No. Connect only the narrow capabilities required by the workflow.*

01 Create the Webhook Entry Point

Start a new workflow with a Webhook node so an external application can submit a support request as JSON.

- Create a new workflow
- Add Webhook trigger
- Use POST
- Set a simple path such as /agent-support
- Run in test/listen mode
- Send a sample JSON payload

Why this matters: A webhook turns the visual workflow into an application endpoint instead of a manual demo.

Try this

```
POST /webhook-test/agent-support
Content-Type: application/json

{
  "ticket_id": "T-204",
  "customer_id": "C-104",
  "message": "My shipment is late. Please update me."
}
```

Expected result

```
# Instructor example — expected output
OUTPUT  The execution panel shows ticket_id, customer_id, and message from the request body.
```

02 Add the AI Agent and Gemini Model

Add an AI Agent node and connect a Google Gemini Chat Model sub-node. Give the agent a clear system message that defines its job and its boundaries.

- Add AI Agent after the Webhook
- Connect Google Gemini Chat Model
- Select/create Gemini credentials
- Map the incoming message into the agent input
- Add a concise system instruction
- Test with no tools first

Why this matters: The model should understand the request before tools are added. Separating basic reasoning from tool integration makes debugging easier.

Try this

```
Suggested system instruction:

You are a customer-support operations agent.
Use connected tools for live customer or order data.
Never invent balances, order status, or contact details.
```

```
Do not send external messages unless the workflow explicitly allows it.
Keep answers concise and factual.
```

Expected result

```
# Instructor example — expected output
OUTPUT The AI Agent produces a response, but it should admit that it cannot know live order data until a tool is connected.
```

03 Add Short-Term Memory

Attach a memory sub-node so a multi-turn chat can retain recent context. Use a session key that is stable for the same ticket or user.

- Add a Simple Memory / buffer-style memory node supported by your n8n version
- Connect it to the AI Agent memory port
- Set the session key to ticket_id or another stable conversation identifier
- Test two turns that refer to the same ticket

Why this matters: Without a stable session identifier, unrelated users can lose context or, worse, share context accidentally.

Try this

```
Session key expression idea:
{{$json.body.ticket_id}}

Turn 1: "My order is late."
Turn 2: "Can you summarize what we know so far?"
```

Expected result

```
# Instructor example — expected output
OUTPUT The second turn can refer to the earlier conversation when the same session key is used.
```

> **PRO TIP** *Stretch: Run a second ticket ID and confirm its memory is isolated from the first ticket.*

04 Use Google Sheets as a Read-Only Tool

Create a small sheet with customer/order data and expose a lookup operation as a tool connected to the AI Agent.

- Create columns such as customer_id, order_id, status, total
- Add the Google Sheets node as an AI tool or tool-connected action
- Configure a read/lookup operation only
- Describe clearly what the tool returns
- Ask the agent about C-104 or order 104

Why this matters: A read-only data tool grounds the model in live business data without granting write permissions.

Try this

```
Example sheet rows:
customer_id | order_id | status   | total
C-104       | 104      | delayed  | 850
C-205       | 205      | shipped  | 420

User: "What is the status of order 104?"
```

Expected result

```
# Instructor example — expected output
OUTPUT  The agent calls the Sheets tool and answers using the returned delayed status and total.
```

PRO TIP *Stretch: Add a second tool for a shipping policy lookup and ask a question that requires both pieces of information.*

05 Connect Slack or Gmail for Delivery

Add one external delivery integration. For a safer classroom workflow, let the agent prepare the content, then gate the actual send action behind review or a separate branch.

- Choose Slack Post Message or Gmail Send/Draft
- Use a fixed test channel/inbox
- Map the AI-generated draft into the message body
- Keep destination parameters constrained
- Add a review step before the real send when possible

Why this matters: Messaging tools create side effects outside n8n. Tool access and credentials should be narrower than a human administrator account.

Try this

```
Safe pattern:
AI Agent → Draft message → Review/approval → Send tool

Avoid:
AI Agent → unrestricted Gmail/Slack action with no review
```

Expected result

```
# example — expected output
OUTPUT  A reviewed test message reaches only the intended test destination.
```

06 Finish the Workflow: Response, Errors, and Activation

Return a clear webhook response, add an error/fallback path, test duplicate requests, then activate the workflow only after the behavior is stable.

- Add Respond to Webhook (or equivalent response path)
- Return status + concise message

- Create an error branch or error workflow
- Log ticket_id and tool outcome
- Test with a missing customer
- Test the same webhook twice
- Only then activate the production webhook URL

Why this matters: A workflow is production-ready only when both the happy path and the failure path are intentional.

Try this

```
Example response body:
{
  "ticket_id": "{{$json.body.ticket_id}}",
  "status": "processed",
  "message": "{{$json.output}}"
}

Error case should return a clear controlled result, not a silent failure.
```

Expected result

```
# example — expected output
OUTPUT  Successful requests return a structured response; missing data follows the designed fallback instead of hallucinating.
```

PRO TIP *Stretch: Add an idempotency check so re-sending the same ticket_id does not send the same external message twice.*

Final Architecture

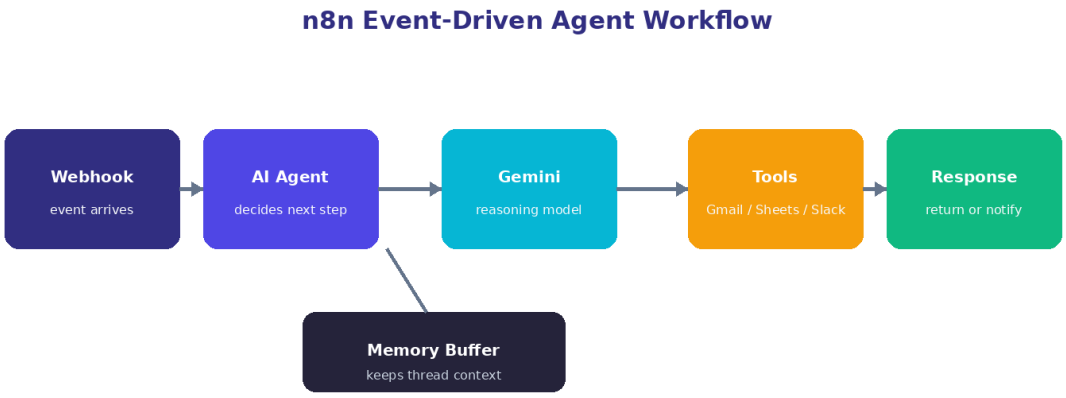

Figure L.1 — Final lab architecture: webhook → agent + Gemini + memory → tools → response/notification.

Congratulations

You completed the three-day progression: native tool-calling agent → stateful graph with human approval → visually orchestrated event-driven agent workflow. You can now choose the implementation style that fits the system instead of treating every agent project the same way.